MIP Engineering Team

SUMMARY

This document defines the official repository structure for the Mainframe Intelligence Platform (MIP). The objective is to ensure consistency, maintainability, scalability, team collaboration, and AI-Assisted Development.

PROJECTS

Repository Structure Standard | Version: 1.0; Status: Approved; Owner: MIP Engineering Team

- Defines the official repository structure for the Mainframe Intelligence Platform (MIP).
- Every contributor must follow this structure.
- No new top-level directories should be created without architecture review.
- The repository structure mirrors the flow: Source Code → Metadata → Relationships → Knowledge Graph → Reasoning → Modernization Intelligence.

TECHNICAL SKILLS

Clean Architecture
Domain Driven Design
Modular Monolith
Plugin-Based Discovery
Metadata First Design
Repository Governance
GitHub Actions
Architecture
Roadmaps
Standards
Playbooks
Decision Records
Leadership Materials
Reusable GitHub Copilot Skills
Reusable Prompt Library
Production code
Repository discovery
File scanning
Artifact classification
Inventory generation
Source code parsing
Metadata management
Metadata creation
Metadata validation
Metadata persistence
Persistence layer
SQLite operations
Future PostgreSQL operations
Knowledge Graph Layer
Node creation
Edge creation

Graph traversal
Graph queries
NetworkX
Application services
Orchestration
Workflow execution
Use case implementation
Impact analysis
Dependency analysis
Flow analysis
Service identification
API identification
Migration recommendations
All automated testing
Automation
Developer utilities
Sample repositories
Deployment assets

HIGH LEVEL REPOSITORY LAYOUT

mip-platform/ with top-level directories: .github, docs, skills, prompts, src, tests, scripts, tools, examples, output, data, logs, deployment.

REPOSITORY OWNERSHIP

docs: Architecture Team; skills: Platform Team; prompts: Platform Team; src: Engineering Team; tests: Engineering Team; scripts: DevOps Team; tools: Platform Team; examples: Engineering Team; output: Runtime; data: Runtime; logs: Runtime; deployment: DevOps Team.

DEPENDENCY RULES

Allowed: api → services → domain, repository, graph. Forbidden: domain → api; parsers → api. Domain must remain independent.

NAMING STANDARDS

Packages: snake_case; Modules: snake_case; Classes: PascalCase; Functions: snake_case; Constants: UPPER_CASE.

BRANCH STRATEGY

main, development, feature/*, bugfix/*, release/*.

DEFINITION OF DONE

A repository is compliant when folder structure matches document, skills installed, prompt library installed, Copilot instructions configured, testing structure created, documentation structure created, and dependency rules enforced.

SUCCESS METRIC

A new engineer can clone the repository, understand the structure, find code quickly, add a feature safely, and use Copilot effectively within one day of onboarding.